

第四章 软件设计 AI讲解

开篇导读

一句话概括：软件设计是把"做什么"（需求）转化为"怎么做"（实现方案）的过程——它分为总体设计（划分模块）和详细设计（设计每个模块的内部逻辑）。

本章在考试中的地位：高频核心章。内聚与耦合的7级排序是几乎必考的送分题，McCabe环形复杂度计算是计算题常客，模块化设计原则是简答题高频。本章坑点密集在"排序方向"和"耦合/内聚类型判断"上。

一、软件设计的两个阶段

核心概念

- 软件设计分为两个阶段：
- 1. 总体设计（概要设计）：确定系统由哪些模块组成，模块间如何联系——回答"系统怎么划分"
 - 2. 详细设计：确定每个模块的具体实现逻辑——回答"模块内部怎么实现"

为什么重要

设计阶段是把需求（逻辑模型）转化为实现方案（物理模型）的桥梁。总体设计关注"结构"（模块划分），详细设计关注"过程"（模块内部算法）。两者层次不同，不能混为一谈。

易混辨析

考法	易错点
"总体设计" vs "详细设计"	总体设计管"模块划分和结构"，详细设计管"模块内部算法"。总体设计输出总体设计说明书，详细设计输出详细设计说明书
"设计" vs "需求分析"	需求分析=做什么（逻辑），设计=怎么做（物理）

记忆技巧

口诀："总体划模块，详细写算法"——总体设计是"划分"，详细设计是"写算法"。

二、设计原理

核心概念

- 软件设计的四大基本原理：
- 1. 模块化：把程序分解成独立命名、可独立访问的模块
 - 2. 抽象：抽出事物的本质特征，忽略非本质细节
 - 3. 信息隐藏：模块内部的信息对其他模块不可见，只通过接口暴露

4. 局部化：把关系密切的元素放在一起

为什么重要

这些原理是评判设计好坏的标准。模块化是手段，抽象和局部化是方法，信息隐藏是目标——最终目的是让模块"高内聚、低耦合"。

四大原理深度拆解

1. 模块化 (Modularity)

- 是什么：把大型软件分解成多个独立命名、可独立访问的"模块"，每个模块完成特定功能
- 为什么必要：

1. 对抗复杂度——人脑一次能处理 7 ± 2 个概念，模块化把万行代码切成可理解的小块

2. 并行开发——多人同时开发不同模块，互不干扰

3. 独立测试——单元测试、修改、替换都以模块为单位

4. 复用能力——好模块可在多个项目复用（如日志模块、加密模块）

- 模块化的"度"问题：
 - 太细碎（如每个函数一个模块）→ 接口爆炸、调用关系复杂
 - 太粗放（如整个系统一个模块）→ 失去模块化意义
 - 最佳点：模块大小50-100行（经验值），单一职责
- Meyer的5条模块化标准：
 - 可分解性：能把大问题拆成独立的子问题
 - 可组装性：模块能像积木拼出新系统
 - 可理解性：单独看一个模块就能明白它做什么
 - 连续性：需求小改动只引起对应模块的小修改
 - 保护性：异常被局部限制在模块内，不传染外部
 - 核心机制：通过函数/类/包/服务等不同粒度实现

2. 抽象 (Abstraction)

- 是什么：抽出事物的本质特征，忽略非本质细节
- 三种抽象层次（从高到低）：

1. 数据抽象：用抽象数据类型（ADT）表示，隐藏数据结构（如用"队列"概念，不关心数组还是链表实现）

2. 过程抽象：用函数/方法名表示一系列操作（如sortList()，不关心是快排还是冒泡）

3. 控制抽象：用控制结构表示控制流（如while、for，不关心底层跳转指令）

- 核心方法：自顶向下，逐步求精——先抽象再具体，避免一开始陷入实现细节
- 抽象的代价：每一层抽象都有性能开销和理解成本
- 判断标准：能在更高层次描述系统行为而不暴露低层细节

3. 信息隐藏 (Information Hiding)

- 是什么：模块内部的信息（数据结构、算法实现）对其他模块不可见，只通过接口暴露必要功能
- Parnas原则 (David Parnas 1972)："模块应该隐藏可能变化的设计决策"——设计决策最易变的部分要藏在模块内

- 机制：
 - OO语言：private/protected访问修饰符
 - 模块系统：export/import控制
 - C语言：static函数/static变量



• 优点：

1. 修改自由——内部实现可改而不影响其他模块

2. 降低复杂度——使用方只需关心接口

3. 减少错误传播——内部bug不易跨模块扩散

• 与封装的关系：封装是OO语言对信息隐藏的实现机制，信息隐藏是更广义的设计原则

4. 局部化 (Localization)

• 是什么：把关系密切的元素（数据、操作）放在同一个模块内，相关性弱的放到不同模块

• 作用：

1. 高内聚的物理基础——相关元素物理上接近，自然形成内聚

2. 降低模块间通信——相关元素在同一模块内通信不需跨接口

3. 便于理解——读代码时不用跨多个文件查找相关逻辑

• 判断标准：

• 相关元素是否经常一起被修改（一起修改→应放一起）

• 相关元素是否共享数据（共享→应放一起）

• 反例："工资计算"逻辑分散在3个模块——员工模块算基本工资、考勤模块算出勤工资、税务模块算扣税；改个工资规则要改3个模块

四大原理的因果关系

四原理不是并列罗列，而是因果链：

恍然大悟：模块化是手段→抽象帮你识别模块边界→信息隐藏定义模块接口→局部化保证模块紧凑——四原理形成一条因果链，最终目的都是"高内聚低耦合"。

>

- 没有抽象，就不知道怎么切模块（边界不清）；

- 没有信息隐藏，模块切了也白切（内部仍被外部依赖）；

- 没有局部化，模块虽切了但东西散落（内聚不高）。

>

所以四原理缺一不可：抽象告诉你"切在哪"，信息隐藏告诉你"露什么"，局部化告诉你"装什么"，模块化是结果。这条因果链是软件工程对抗复杂度的核心武器，从需求分析（DFD分层）、需求建模（OO封装）到详细设计（控制结构）、测试（单元测试），每个阶段都在用这4个原理。理解了这4个原理，就理解了"软件工程为什么这么搞"。

记忆技巧

口诀："模抽隐局"（模块化、抽象、信息隐藏、局部化）。

三、模块独立性 核心考点

核心概念

模块独立性是衡量模块质量的标准，用两个指标度量：

• 内聚 (Cohesion)：模块内部各元素结合的紧密程度——越高越好

• 耦合 (Coupling)：模块之间联系的紧密程度——越低越好

口诀："内聚高，耦合低"——这是模块设计的黄金法则。

内聚的7种类型（从低到高，必背！）



级别	名称	含义	通俗解释
1 (最低)	偶然内聚	模块内各成分无关系，只是碰巧放一起	"硬凑"
2	逻辑内聚	各成分逻辑相似，由参数选择执行哪个	"同类但各干各的"
3	时间内聚	各成分在同一时间段执行	"同时干活"
4	过程内聚	各成分按特定顺序执行	"按步骤来"
5	通信内聚	各成分操作同一数据集	"共享数据"
6	顺序内聚	一个成分的输出是下一个的输入	"流水线"
7 (最高)	功能内聚	所有成分共同完成一个单一功能	"一心一意"

记忆口诀：偶逻时过通顺功（偶然→逻辑→时间→过程→通信→顺序→功能，从低到高）

7种内聚的具体例子（帮你想通为什么功能内聚最好）：

内聚类型	具体场景例子
偶然内聚	一个模块里塞了"求平方根+打印报表+读记录"三个无关功能（纯粹硬凑）
逻辑内聚	一个模块既能求max、又能求min、又能求avg，由传入flag决定执行哪个（相似功能flag切换）
时间内聚	初始化模块：同时"打开文件+初始化变量+加载配置"（同一时刻执行，但功能无关）
过程内聚	模块按"读记录→校验→计算"顺序执行（有先后步骤，但前一步输出不是后一步输入）
通信内聚	模块里"读订单+改订单+删订单"都操作同一份订单数据（共享数据集）
顺序内聚	模块"读记录→校验记录→计算结果→输出"，前一步输出是后一步输入（流水线）
功能内聚	模块只做"计算订单总价"一件事，所有代码都为这一个功能服务（单一职责）

恍然大悟：为什么功能内聚最好？因为功能内聚的模块"只做一件事"——这就像一把只切菜的菜刀（坏了换一把，不影响切肉的工具）；而偶然内聚像一个杂物箱什么都往里扔（动一样牵动其他）。修改功能内聚模块不会牵连其他功能，这就是"高内聚"的力量。内聚越高，模块越独立，维护越安全。

耦合的7种类型（从高到低，必背！）

级别	名称	含义	通俗解释
1 (最高，最差)	内容耦合	一个模块直接访问另一个模块的内部内容	"硬闯别人家"
2	公共耦合	多个模块共享同一个全局数据结构	"共用一个大仓库"
3	外部耦合	模块访问同一全局简单变量（非数据结构）	"共用一个开关"



4	控制耦合	模块间传递控制信息（如flag）	"发号施令"
5	特征耦合	传递整个数据结构（但只用了其中一部分）	"给多了"
6	数据耦合	传递简单数据参数	"正好够用"
7（最低，最好）	非直接耦合	模块间无直接联系，通过主模块控制	"互不干涉"

记忆口诀：内公外控特数非（内容→公共→外部→控制→特征→数据→非直接，从高到低）

7种耦合的具体例子（帮你想通为什么数据耦合好而控制耦合差）：

耦合类型	具体参数传递例子
内容耦合	模块A直接读写模块B的全局变量 moduleB.x = 10（硬闯别人家改东西）
公共耦合	多个模块都访问同一个全局数据结构 global OrderList（共用大仓库，谁都改，出问题找不到谁）
外部耦合	两个模块都访问同一个全局简单变量 global flag（共用一个开关）
控制耦合	模块A传 flag=1 给模块B，B根据flag决定执行排序还是查找（A指挥B怎么做）
特征耦合	模块A传整个Student记录给模块B，但B只用其中的age字段（给多了，B不该看到其他字段）
数据耦合	模块A只传 age=18 给模块B（正好够用，B只收到需要的数据）
非直接耦合	模块A和模块B没有直接调用，都由主模块分别调用（互不干涉）

恍然大悟：为什么数据耦合好而控制耦合差？因为数据耦合是"我给你原料你自己做"（A给age，B自己决定怎么用），接收方自主；控制耦合是"我指挥你怎么做"（A给flag指挥B），接收方依赖发送方的指挥——改指挥逻辑就得改接收方，牵一发动全身。所以数据耦合松散、控制耦合紧密。

设计原则

- 尽量使用数据耦合（好的耦合）
- 少用控制耦合
- 限制公共耦合的范围
- 完全不用内容耦合（最差）

易混辨析（重点坑点）

考法	易错点
"数据耦合是高耦合吗？"	不是！数据耦合是低耦合（第6级，仅次于最好的非直接耦合）。数据耦合是我们希望的耦合方式
"控制耦合传递什么？"	传递控制信息（如flag、开关），不是数据。这是区分控制耦合和数据耦合的关键
"特征耦合传递什么？"	传递整个数据结构，但接收方只用一部分。"给多了"是特征耦合的标志



"内聚从低到高还是从高到低？"	内聚从低到高：偶然（最低）→功能（最高）。耦合从高到低：内容（最高）→非直接（最低）。两者方向相反！
-----------------	--

记忆技巧

内聚口诀："偶逻时过通顺功"——谐音"偶尔经过通顺功"：偶尔（偶然）经（逻辑）过（时间）通（过程）顺（通信）功（顺序功能）。从低到高。

耦合口诀："内公外控特数非"——从高到低。谐音"内公外控特数非"。

核心规律：内聚越高越好，耦合越低越好。

四、启发式规则

核心概念

启发式规则是指导模块设计的经验法则：

1. 改进软件结构：提高内聚，降低耦合
2. 模块规模适中：模块太大难理解，太小接口太多
3. 扇出适中（3-4）：一个模块直接调用的下级模块数
4. 扇入越大越好：调用本模块的上级模块数（复用性高）
5. 控制域包含作用域：模块的控制域（自身+直接下属）应包含其作用域（受其判断影响的模块）
6. 单入口单出口：模块只有一个入口和一个出口
7. 功能可预测：相同输入产生相同输出

扇入 vs 扇出（易混点）

概念	方向	建议
扇出 (Fan-out)	向下（调用多少个下级）	适中（3-4），不要太多
扇入 (Fan-in)	向上（被多少个上级调用）	越大越好（复用性高）

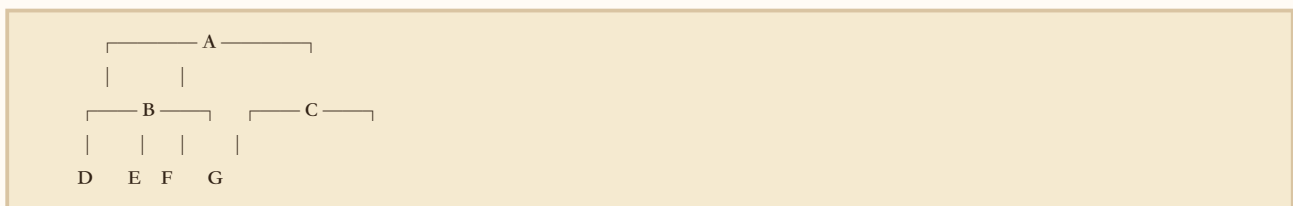
控制域 vs 作用域（必背）

这是启发式规则中最容易考但最难理解的一对概念：

- 控制域 (Scope of Control)：一个模块自身 + 它直接或间接调用的所有下属模块
- 作用域 (Scope of Effect)：一个模块中判断 (if) 所影响的其他模块——即根据这个判断结果决定要不要调用某些模块

核心原则：作用域应在控制域内——即"我做的判断只能影响我管得到的人"。

调用层次图示例：



例1（合规）：B中有判断"if x>0 then 调E"



- B的控制域 = {B, D, E}
- 判断的作用域 = {E} (E受判断影响)
- 作用域{E} 控制域{B,D,E} 合规

例2 (违规): B中有判断"if x>0 then 调F"

- B的控制域 = {B, D, E}
- 判断的作用域 = {F} (F受判断影响)
- 作用域{F} 控制域{B,D,E} 违规——B管不到F但试图影响F

违规后果:

- B的判断需要把结果传给A, A再传给C, C再决定调F——形成跨模块的复杂调用链
- 改一个判断要改多个模块——耦合度暴增

修复方法: 把判断从B移到A (A的控制域包含{A,B,C,D,E,F,G}, 足够覆盖F)

单入口单出口 (结构化基础)

- 要求: 每个模块只有一个入口 (唯一进入点)、一个出口 (唯一退出点)
- 为什么:

1. 多入口/多出口的模块跳转混乱 (goto泛滥)

2. 难以单元测试 (不知道从哪进、从哪出)

3. 违反结构化定理 (任何程序都可由顺序/选择/循环组合而成, 不需要goto)

- 历史: Dijkstra 1968年发表《GOTO Statement Considered Harmful》, 从此结构化编程成为主流

模块规模的经验值

- 推荐: 50-100行
- 太大 (>200行): 单一职责被破坏, 难理解、难测试
- 太小 (<10行): 可能拆得过细, 接口数量爆炸
- 判断方法: 能否用一句话描述模块功能 ("做X"), 不能则需拆分; 接口参数>5个考虑拆模块或封装参数对象 (参数过多通常是数据耦合恶化的信号)

恍然大悟: 7条启发式规则其实都是"高内聚低耦合"的具体实现手段——模块规模适中 (避免一个模块管太多→低内聚)、扇出适中 (避免一个模块依赖太多→高耦合)、扇入越大越好 (鼓励复用, 反映高内聚的好模块)、控制域包含作用域 (避免跨模块判断→低耦合)、单入口单出口 (避免跳转混乱→高内聚)、功能可预测 (无副作用→高内聚)。记不住7条没关系, 记住"高内聚低耦合"这8个字就能推出7条。这也是为什么内聚耦合是软件设计的"宪法", 启发式规则是宪法下的"细则"。

记忆技巧

口诀: "扇出适中3-4, 扇入越大越好"——扇出是"管几个孩子" (别太多), 扇入是"几个爹叫你" (越多说明你越有用)。

五、图形工具

核心概念

1. 层次图: 用树形结构表示模块间的调用关系

2. HIPO图: 层次图+输入处理输出表

3. 结构图: 精确描述模块间的调用关系, 符号包括:

- 模块 (矩形)
- 调用 (箭头)



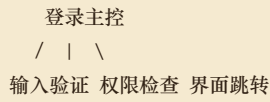
- 数据（带空心圆的箭头）
- 控制信号（带实心圆的箭头）

三种图的演进对比（细节递增）

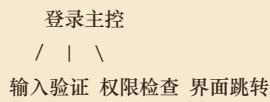
三种图是同一系统的"细节递增"——从骨架到血肉：

登录系统的三层图对比：

【层次图】只画调用骨架



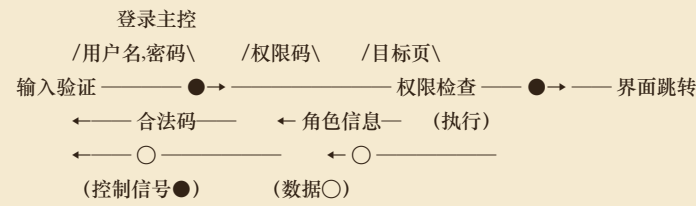
【HIPO图】层次图 + IPO表（每个模块加输入/处理/输出说明）



IPO表（输入验证模块）：

- 输入：用户名、密码
- 处理：检查格式合法性
- 输出：合法标志（true/false）

【结构图】层次图 + 数据流 + 控制信号



易混辨析

工具	特点
层次图	只显示调用关系，不显示数据流
结构图	显示调用+数据+控制信号（更详细）
HIPO图	层次图+IPO表（带处理细节）

三种图何时用哪个

阶段	推荐工具	原因
总体设计早期（草拟架构）	层次图	只看调用骨架，避免陷入细节
设计评审/沟通用户	HIPO图	加IPO表说明每个模块"做什么"，业务人员看得懂
详细设计/编码前	结构图	完整描述调用+数据+控制，开发可直接参考

恍然大悟：三种图是"细节递增"的同心圆——层次图骨架（只有调用）→ HIPO图加内容（IPO表说明做什么）→ 结构图加数据（完整数据流和控制流）。同一个系统，不同阶段画不同图，避免一开始就



陷入数据流细节（早期不知道详细数据，画结构图反而误导）。这与第二章DFD分层、第三章用例图扩展流程是同一思路：软件工程的图永远是从粗到细，避免过早绑定细节。

六、面向数据流设计

核心概念

根据数据流图的类型，把DFD映射为软件结构：

- 变换流：数据沿输入路径进入，经过变换处理，沿输出路径流出（输入→变换→输出）
- 事务流：数据到达一个事务中心，根据类型分派到不同的处理路径

两种数据流深度拆解

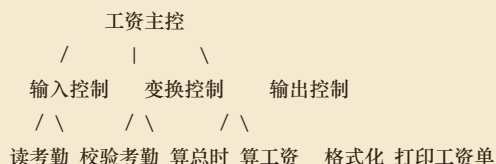
1. 变换流 (Transform Flow)

- 是什么：数据流呈"流水线"形态——输入数据进入系统→经过若干变换处理→输出结果
- DFD特征：流程图呈"线性管道"，没有显著的分支决策点
- 典型例子：工资计算系统——考勤数据 → 计算总时数 → 计算工资 → 计算扣税 → 输出工资单

变换分析的5步映射（DFD → 结构图）：

1. 找输入流的最高抽象点——输入数据从外部进入系统后第一次被加工的位置
2. 找输出流的最高抽象点——输出数据被变换为最终形式前的最后位置
3. 划定3个边界——输入流（左侧）、变换中心（中间）、输出流（右侧）
4. 设计主控模块——位于结构图顶层，调用三个一级模块
5. 设计三层结构：
 - 第一级：输入控制、变换控制、输出控制
 - 第二级：每个一级模块下的具体处理模块

变换流结构图示例：



2. 事务流 (Transaction Flow)

- 是什么：数据流到达一个事务中心后，根据事务类型分派到不同的处理路径
- DFD特征：流程图中有一个明显的分派点（事务中心），后接多条平行路径
- 典型例子：订单处理系统——订单到达 → 识别订单类型 → 分派到（普通订单/团购订单/秒杀订单/退款订单）的不同处理流程

事务分析的4步映射（DFD → 结构图）：

1. 识别事务中心——DFD中数据流分派的关键节点
 2. 设计调度模块（事务控制模块）——位于结构图顶层，根据事务类型分派
 3. 设计每条事务路径——每种事务类型对应一个分支
 4. 每条路径内部用变换分析——具体处理细节再用变换流方法展开
- 事务流结构图示例：



订单调度
 / | | \
 普通订单 团购订单 秒杀订单 退款订单
 /\ /\ /\ /\
 校验 入库 抢货 锁库 验时 抢库 退货 退款

变换流 vs 事务流 选用判断

特征	变换流	事务流
DFD形状	线性管道 (→→→→)	中心+辐射 (→★)
核心结构	输入→变换→输出 三段式	事务中心 + 多条分支
关键决策	找输入/输出的"最高抽象点"	找"事务中心"和分类逻辑
类比	工厂流水线 (按顺序加工)	分拣中心 (按类型分发)

实际项目通常组合使用：顶层是事务流（按业务类型分派），每条业务分支内部是变换流（顺序处理）。

恍然大悟：变换流和事务流的本质区别——变换是"流水线"（按顺序加工），事务是"分拣中心"（按类型分发）。判断方法：看DFD的形状！如果像水管（线性流动），用变换分析；如果像放射星（一点分多支），用事务分析。DFD的形状决定了用哪种映射方法——这是面向数据流设计的精髓。所以SA阶段画好DFD后，到SD阶段不需要重新分析，看图就能选方法。这也是为什么SA和SD能配套（合称SA/SD方法）——它们用同一种"数据流"语言，从需求自然过渡到设计。

记忆技巧

口诀："变换三段式（输入→变换→输出），事务分派式（中心→多路径）"。

判断口诀："管道用变换，分发用事务"——DFD像管道用变换流方法，DFD像分拣中心用事务流方法。

七、详细设计工具

核心概念

详细设计阶段描述模块内部逻辑的工具：

1. 结构程序设计：只用三种基本控制结构（顺序、选择、循环）
2. 判定表：用表格表示复杂的多条件组合逻辑
3. 判定树：用树形图表示条件判断逻辑
4. PDL（过程设计语言/伪代码）：用类自然语言描述算法

三种基本控制结构（必背）

- 顺序：按先后顺序执行
- 选择：根据条件选择执行路径（if-else）
- 循环：重复执行某段代码（while/for）

结构化定理（Böhm-Jacopini定理）

定理内容：任何程序都可以由顺序、选择、循环这三种基本控制结构组合而成，不需要goto语句。

意义：

- 1966年Böhm和Jacopini证明，奠定了结构化编程的理论基础



- Dijkstra据此提出"GOTO Statement Considered Harmful", 从此goto被禁用
- 现代编程语言 (Java、Python等) 的控制结构都基于这三种

为什么禁用goto:

- goto让程序流转混乱 ("意大利面条代码"), 难以理解和维护
- 三种结构化控制流就足以表达任何算法, 不需要goto
- 单入口单出口原则的实现基础

详细设计工具深度对比

1. PDL (伪代码)

- 是什么: 用接近自然语言的语法描述算法, 无具体编程语言细节
- 例子: 求数组最大值

```
IF 数组为空 THEN
    返回错误
ELSE
    max ← 第一个元素
    FOR 每个元素 e IN 数组:
        IF e > max THEN
            max ← e
        ENDIF
    ENDFOR
    返回 max
ENDIF
```

- 优点: 自然语言便于沟通; 不绑定具体语言; 易转换为代码
- 缺点: 无标准 (每个团队的PDL方言不同); 逻辑复杂时仍可读性差
- 适用: 算法描述、流程文档化

2. 判定表 (Decision Table)

- 是什么: 用表格穷举所有"条件组合"和对应"动作"
- 结构: 上半部分=条件桩+条件项, 下半部分=动作桩+动作项
- 例子: 保险费率 (4种条件组合)

条件	规则1	规则2	规则3	规则4
年龄 ≥ 30	Y	Y	N	N
男性	Y	N	Y	N
动作				
收基本保费	✓	✓	✓	✓
加成20%	✓			
加成30%			✓	

- 优点: 条件组合穷举 (不漏情况); 适合复杂多条件逻辑
- 缺点: 条件多时表很大 (5个条件就32列); 规则间冗余难发现
- 适用: 业务规则复杂、条件组合多 (如保险、财务、风控)

3. 判定树 (Decision Tree)

- 是什么: 用树形图表示条件判断逻辑, 节点是判断, 叶子是动作
- 例子 (同保险费率):





- 优点：直观（层次清晰）；条件少时易读
- 缺点：条件多时树过大（指数级节点）；不易穷举所有组合（容易漏分支）
- 适用：条件层次明确（如先按年龄分大类、再按性别分小类）

三种工具的选用对比

工具	表达力	适用场景	风险
PDL	算法描述（强）	复杂算法、循环嵌套	自由度高，团队规范要严
判定表	穷举条件组合	多条件业务规则	条件多时表爆炸
判定树	层次化判断	条件有明确优先级	易漏分支

实际项目通常组合使用：业务规则用判定表（穷举不漏）→ 转判定树（层次化便于理解）→ PDL实现（伪代码翻译）。

恍然大悟：详细设计工具按"表达力"分两类——图形化（判定表/树）适合复杂逻辑（多条件穷举），文本化（PDL）适合算法描述（顺序流程）。判定表和判定树是同一逻辑的两种表达：表是"穷举型"（看一眼所有组合），树是"分支型"（按优先级走分支）。条件多用表（怕漏），层次清用树（怕乱）。三种工具+三种基本控制结构构成了详细设计的"工具箱"，覆盖了所有逻辑表达需求——这也是为什么1966年的结构化定理至今有效，三种控制结构+几种工具足以表达任何程序。

八、McCabe环形复杂度 计算题必考

核心概念

McCabe环形复杂度用图论方法度量程序的复杂度，公式：

$$V(G) = E - N + 2$$

其中：

- E = 程序流程图中的边数（箭头数）
- N = 程序流程图中的节点数（方框数）
- V(G) = 环形复杂度

为什么重要

这是计算题必考点！给出一个程序流程图，要求计算V(G)。V(G)还等于独立路径数，即至少需要多少个测试用例才能覆盖所有独立路径。

计算步骤

1. 画出程序流程图
2. 数清楚节点数N和边数E



3. 套公式 $V(G) = E - N + 2$

三种等价公式（结果一致，可互相验证）

公式	含义	适用场景
$V(G) = E - N + 2$	边数-节点数+2（边点法）	能数清边和节点时
$V(G) = P + 1$	判定节点数+1（判定节点法）	能数清if/while/for判定节点时
$V(G) = \text{区域数}$	图中划分的区域数（含图外部区域）	画图后能数清封闭区域时

完整算例

代码：

```
1. 输入 A, B

2. if (A > 0)

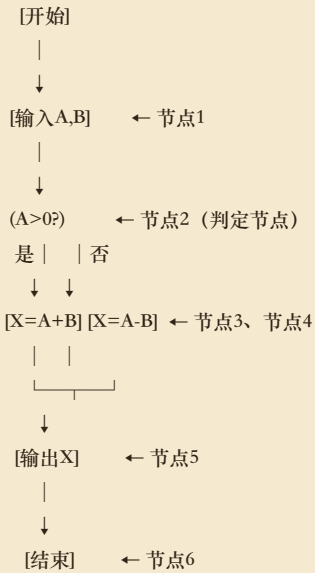
3.   X = A + B

4. else

5.   X = A - B

6. 输出 X
```

流程图（ASCII）：



数节点和边：

- 节点数 $N = 7$ （开始、输入、判定、 $X=A+B$ 、 $X=A-B$ 、输出、结束）
- 边数 $E = 7$ （开始→输入、输入→判定、判定→ $X=A+B$ 、判定→ $X=A-B$ 、 $X=A+B$ →输出、 $X=A-B$ →输出、输出→结束）



输出、输出→结束)

三种公式验证 (结果必须一致) :

- 边点法: $V(G) = E - N + 2 = 7 - 7 + 2 = 2$
- 判定节点法: $P=1$ (只有A>0一个判定) , $V(G) = P + 1 = 1 + 1 = 2$
- 区域数法: 图中1个封闭区域 + 1个外部区域 = 2

三种公式都得2, 互相验证

注: 三种公式结果必须一致, 不一致说明数错了。考试时用任意一种都行, 建议用判定节点法最简单 (数if/while/for有几个, 加1) 。

恍然大悟: $V(G)$ 为什么等于独立路径数? 因为每多一个判定节点 (if/while/for) , 就多分出一条独立分支路径——所以判定越多, 独立路径越多, $V(G)$ 越大。这就是判定节点法 $V(G)=P+1$ 的直觉。 $V(G)$ 本质是"这个程序有多少条互不重叠的路径", 所以它就是要测的最少用例数。

易混辨析

考法	易错点
" $V(G)$ 等于什么"	$V(G) = E - N + 2$, 不是 $N - E + 2$, 也不是 $E - N + 1$
" $V(G)$ 代表什么"	代表独立路径数, 即最少需要的测试用例数 (白盒测试基本路径法)

记忆技巧

公式口诀: "边减点加二" ($E - N + 2$) 。

考点聚焦

高频考点清单

1. 内聚7种从低到高排序 (偶选时过通顺功)
2. 耦合7种从高到低排序 (内公外控特数非)
3. McCabe环形复杂度计算 $V(G)=E-N+2$
4. 内聚vs耦合的方向 (内聚高好, 耦合低好)
5. 扇出vs扇入的方向和建议值
6. 总体设计vs详细设计的任务区分
7. 数据耦合是低耦合 (不是高耦合!)
8. 控制耦合传递控制信息 (不是数据)
9. 三种基本控制结构

常见题型

- 排序题: 内聚/耦合的7级排序
- 计算题: McCabe复杂度
- 判断题: "数据耦合是高耦合" (错)
- 场景题: 给定模块关系判断耦合/内聚类型



记忆口诀总览

1. 设计两阶段："总体划模块，详细写算法"
 2. 设计原理："模抽隐局"
 3. 内聚7级（低→高）："偶逻时过通顺功"
 4. 耦合7级（高→低）："内公外控特数非"
 5. 核心法则："内聚高，耦合低"
 6. 扇出扇入："扇出适中3-4，扇入越大越好"
 7. McCabe公式："边减点加二"
 8. 三种控制结构："顺选循"
-

本章小结

本章核心逻辑链：需求（做什么）→总体设计（划模块、定结构）→详细设计（写算法、定逻辑）→编码。

本章最重要的考点是内聚和耦合的7级排序——这是几乎每届必考的送分题，只要背下口诀"偶逻时过通顺功"和"内公外控特数非"就能拿分。其次是McCabe环形复杂度计算（边减点加二），这是计算题的常客。

最大的坑点是"方向"问题：内聚从低到高（偶→功），耦合从高到低（内→非），两者方向相反。还有"数据耦合是低耦合"这个反向判断——数据耦合是我们希望的，不是坏的。记住"内聚越高越好，耦合越低越好"这个黄金法则，本章就拿下了大半。

